

Modulo 1: Nivel de aplicacion. Tema 1: Sockets y Conexiones

Introduccion

Estandares de Internet:

- RFC: Request For Comments
- IETF: Internet Engineering Task Force

¿Que es un protocolo?

Un protocolo define las reglas para:

- Mensajes que deben ser enviados
- Acciones que deben tomarse cuando se recibe un mensaje especifico Los protocolos definen el formato y orden de los mensajes enviados y recibidos entre dispositivos en comunicacion, asi como las acciones a realizar

Paradigma Cliente-Servidor

Servidor:

- Maquina siempre disponible
- Direccion IP permanente
- Normalmente ubicada en centros de datos
- El proceso servidor espera a ser contactado y responde solo cuando se le contacta

Cliente:

- Cualquier dispositivo
- Se conecta de forma intermitente
- Puede (suele) tener una IP dinamica
- Los clientes no se comunican entre si (si hay comunicacion entre dos clientes normalmente sera usando un servidor como proxy)
- El proceso cliente inicia la comunicacion

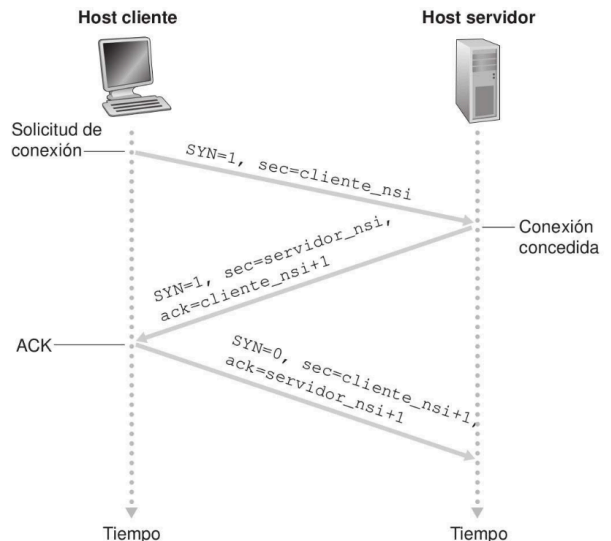
Comunicacion entre procesos

Proceso: programa en ejecucion

- En un mismo sistema se comunican utilizando primitivas (IPC) (semaforos, memoria compartida, paso de mensajes)
- En diferentes sistemas utilizan conexiones por las que intercambian mensajes

Protocolo de conexión cliente-servidor

1. **Cliente manda segmento SYN.** Este tipo de segmento sirve para iniciar la conexión y tiene el campo SYN en la cabecera TCP a 1. Por seguridad, el valor de `cliente_nsi` se calcula inicialmente como un número aleatorio. NO lleva aún los datos que queremos mandar, ya que estamos haciendo el emparejamiento.
2. **El servidor recibe el segmento SYN.** Prepara los buffers y variables TCP para la conexión. Prepara el segmento **SYN-ACK**, de respuesta. Este segmento tampoco lleva datos de aplicación.
3. **El cliente recibe el SYN-ACK.** Prepara sus buffers y variables TCP para la conexión. Envía un último segmento de confirmación, con `SYN = 0` porque la conexión ya ha sido establecida. Este segmento SÍ puede llevar datos.



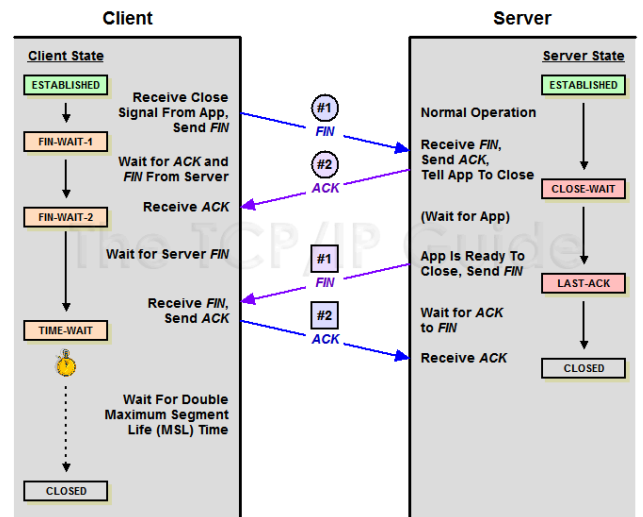
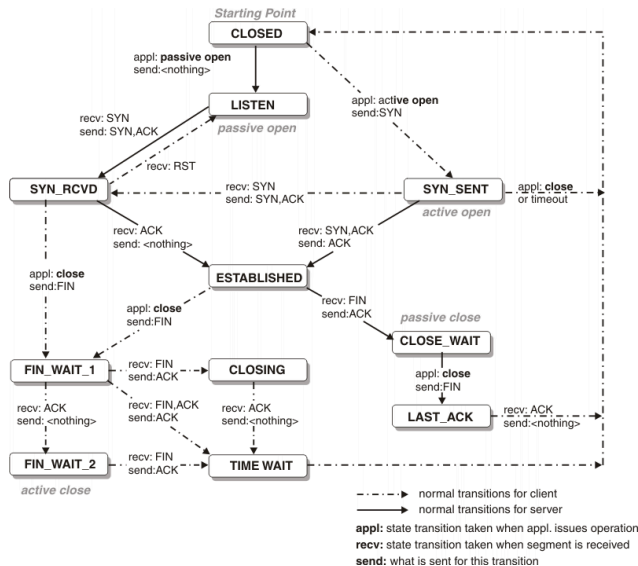
Puertos habituales

Para poder recibir mensajes, un proceso debe tener un identificador unico en esta maquina. Identificador compuesto de:

- Direccion IP (IPv4 32 bits, IPv6 128 bits)
- Numero de puerto: 1 - 65545 (< 1024 privilegiados)

21	FTP
22	SSH
25	SMTP
80, 443	HTTP, HTTPS

Ciclo de vida de un socket



Tanto el cliente como el servidor pueden cerrar la conexión enviando el primer Utilidad de **TIME_WAIT**: esperar el suficiente tiempo para estar seguro de que el servidor ha recibido el ultimo ACK, en caso de que el servidor le comunique que no lo ha recibido, se reenviara

El comando netstat es un comando de Linux para conocer el estado de los sockets y conexiones del SO

```
netstat -a
```

ACT-T1-01 - Escaneando puertos

Hacer escaneos nmap para ver los puertos abiertos en la subred, usar antes ifconfig -a para encontrar la subred (obtener el campo netmask) para convertirla a CIDR Se comprueba cuantas maquinas hay "levantadas" en una red, con el siguiente comando

```
In [ ]: !nmap -sP 192.168.1.136/24 # (en el caso de mi subred ya que netmask = 255.255.255.0)
```

```
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-04-16 16:59 CEST
Nmap scan report for _gateway (192.168.1.1)
Host is up (0.0010s latency).
Nmap scan report for 192.168.1.128
Host is up (0.0011s latency).
Nmap scan report for 192.168.1.130
Host is up (0.0061s latency).
Nmap scan report for 192.168.1.131
Host is up (0.028s latency).
Nmap scan report for alejandroubuntu (192.168.1.136)
Host is up (0.000089s latency).
Nmap scan report for 192.168.1.150
Host is up (0.030s latency).
Nmap scan report for 192.168.1.153
Host is up (0.045s latency).
Nmap scan report for alejandroubuntu (192.168.1.163)
Host is up (0.000067s latency).
Nmap done: 256 IP addresses (8 hosts up) scanned in 2.29 seconds
```

Una vez identificada la maquina de interes, se la puede escanear con:

```
sudo nmap -sS -F -O -sV <<dirIp>>
Parametros utilizados:
```

- Parametros que requieren privilegios (sudo):
 - sS: escaneo SYN, Opcion por defecto si se utiliza "sudo"
 - O: para dar informacion del sistema operativo destino
- Parametros que no requieren privilegios:
 - F: escanea solo los puertos habituales (< 1024), en lugar de los 65545 posibles
 - sV: muestra los servicios de los puertos detectados como abiertos

In [11]: `!nmap -F -sV 192.168.1.136 # En este caso escaneamos mi IP (sin permisos de root)`

```
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-04-16 17:01 CEST
Nmap scan report for alejandroubuntu (192.168.1.136)
Host is up (0.00015s latency).
Not shown: 97 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.9 (Ubuntu Linux; protocol 2.0)
25/tcp    open  smtp     Postfix smtpd
80/tcp    open  http     Apache httpd 2.4.58 ((Ubuntu))
Service Info: Host: alejandroubuntu; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/s
ubmit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.22 seconds
```

1. Modos posibles de un puerto:

- **Abierto:** en el servidor existe un servicio escuchando en ese puerto (el cliente recibe el paquete ACK)
- Cerrado: no hay nada escuchando en el puerto (el cliente recibe de vuelta una señal RST)
- **Filtrado:** hay algo escuchando en el puerto, pero un Firewall (o similar) esta bloqueando el acceso (el cliente no recibe nada de vuelta)

2. Escaneo de una conexion completa vs SYN

Como hemos visto, el protocolo de conexión consta de tres fases. Para saber si hay un puerto abierto, nmap puede realizar dos tipos de escaneo:

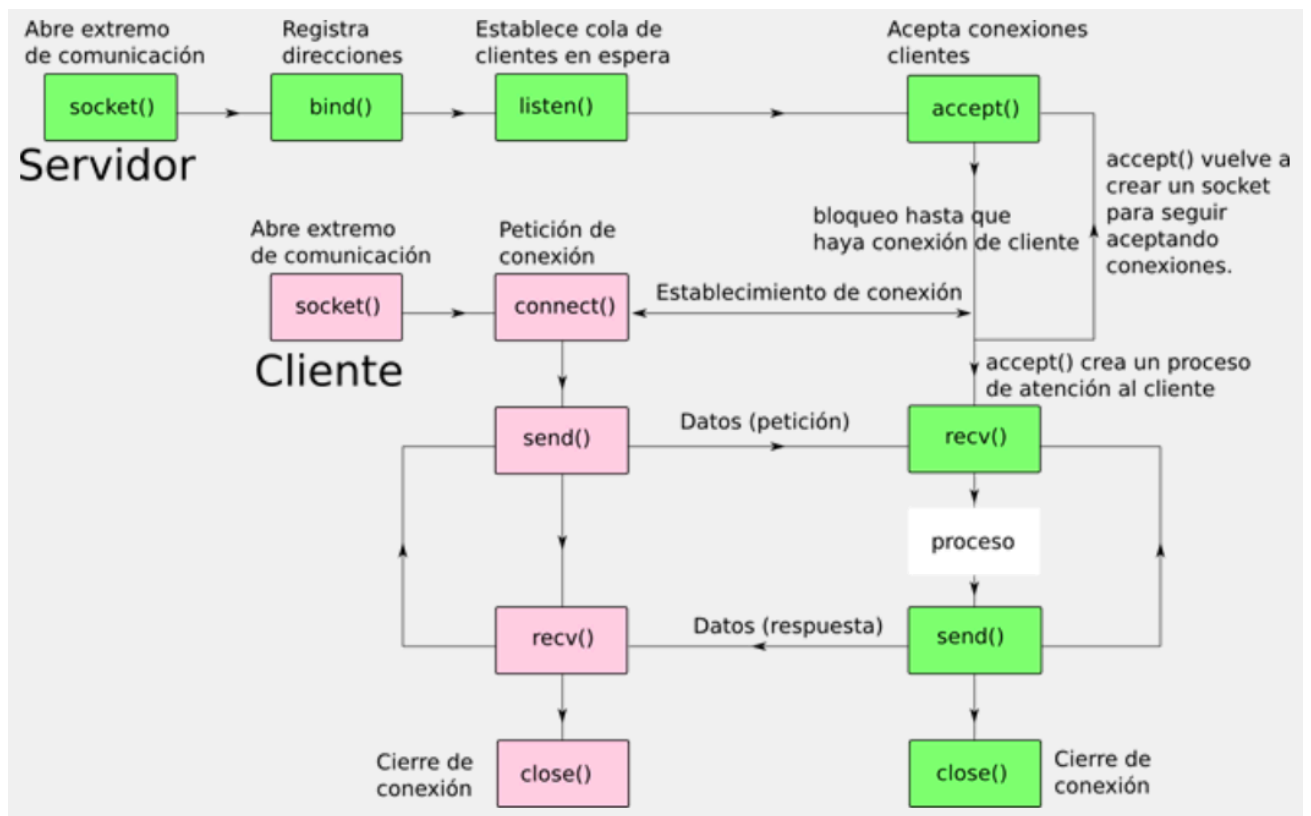
- **Conexion completa:** realiza el protocolo de conexion hasta el final. Es la opcion por defecto si se ejecuta nmap sin sudo
- **SYN:** nada mas recibir el paquete ACK ya se sabe que ese puerto esta abierto, así que corta la conexion (manda RST al servidor) para seguir buscando otros puertos. Es decir, es mas eficiente

Programacion de sockets

TCP y UDP se encargan de la gestion de los sockets para enviar el paquete no solo al destinatario correcto, sino tambien al servicio (corriendo en un puerto) correcto. UDP es un protocolo no confiable entre emisor y receptor ya que los datagramas pueden perderse o llegar desordenados.

Programacion de sockets TCP El protocolo TCP es mas ampliamente usado y sigue los mismos pasos de conexión que UDP. En TCP, antes de empezar a enviarse la informacion, es necesario realizar el emparejamiento que ya se ha comentado. Una vez realizado el emparejamiento, el cliente simplemente enviara los datos por su socket. Esto es distinto a UDP, en donde tenemos que “pegar” una direccion de destino (ip + puerto) antes de enviarlo al socket.

Para poder realizar el emparejamiento necesitamos una cosa fundamental: el servidor debe tener un socket especial para precesar los intentos de emparejamiento. Lo llamaremos “socket de bienvenida” Con esto en mente, los pasos que siguen tanto cliente como servidor TCP son los siguientes:



El socket de conexión mencionado anteriormente es el que se encarga de hacer los pasos hasta `accept()`; cuando llega una conexión nueva, esta se delega en un nuevo socket para realizar las acciones necesarias.

Tipos de servidores

